AI LAB · PROJECT 1E · COURSE STUDY ASSISTANT

StudyMate

A course tutor that answers only from the lecture slides, shows the lecture and slide number, detects the topic, trains students with hints and tells them what to study next.

80.9%	90.4%	91.5%	47 / 50
Right slide first (Top-1), 94 test questions	Right slide in the first three (BM25 Top-3)	Topic classifier accuracy (Logistic Regression)	Off-topic questions refused ("Not found")

Student	**Ahmed Esam Yahia (Ahmed Hallawa)**
Programme	Mechatronics, Systems and AI Automation — Faculty of Engineering, Benha University

Course used as data	Artificial Intelligence (7 lectures, 200 slide passages) + Automotive Engineering MAM 336 (second course)
Real partner	The course lecturer (permission to use the slides, review of topics and questions)
Live site	https://ahmedhallawa.pythonanywhere.com
Source code	https://github.com/pikachu8240/StudyMate (branch `main`)
Date	12 November 2026

Contents

1 Executive summary
2 Problem, users and goals
3 Requirements checklist
4 System architecture
5 Data and data card
6 The AI models
7 Evaluation and results
8 Improvement round (12 Nov)
9 The application — student side
10 The application — lecturer and admin side
11 Excel database and dashboard
12 Google Colab notebooks
13 Security and privacy
14 Testing, deployment and development history
15 Limitations and future work
16 Submission package and how to run

> **How to read this report.** Sections 1–3 give the short version. Sections 5–8 explain the data, the models and every number. Sections 9–11 show the working application with screenshots. Section 16 lists what is inside the ZIP file and how to run it.

1 Executive summary

Before exams students often do not know which topics they really understand, and general chatbots give confident answers that do not match the course slides. **StudyMate** is a Flask web application that works as a tutor for one course at a time. It answers a question **only from the lecture slides**, shows the three closest slides with their lecture and slide number, and says **"Not found in the course"** when the slides do not cover the question instead of inventing an answer.

Behind the pages there are three models: a **search engine** (TF-IDF + cosine similarity, with BM25, LSA and a hybrid as extra methods), a **topic classifier** (Naive Bayes and Logistic Regression) that names the topic of each question, and a **mastery model** that updates a student's level per topic after every practice answer and suggests what to study next.

200	466	94	92
slide passages from 7 lecture PDFs	labelled student-style questions (12 topics)	hand-written search test questions	automatic tests, all passing

Main results (AI course)

- **Topic classifier:** Logistic Regression 91.5% accuracy (macro-F1 0.913) on 94 held-out questions; Naive Bayes 84.0%. Five-fold cross-validation: 91.2% for both.
- **Search:** TF-IDF finds the right slide first for 80.9% of the 94 test questions; BM25 has it in the first three for 90.4%.
- **"Not found":** all 10 off-topic questions of the project sheet are refused; on the extended set 47 of 50 are refused, and 22 of 25 on the half that was never used for tuning.
- **Robustness:** the classifier keeps 89% with typos and 100% on Arabic questions.

Beyond the minimum

The application is deployed on PythonAnywhere, works in English and Arabic (interface and questions), supports several courses and several lecturers (each lecturer sees only his own courses and students), and adds exams, a study studio, a study plan, a weekly report, chat with requests and blocking, notifications, a calendar, badges, an activity log and two Google Colab notebooks for training and comparing models.

> **Still to do before submission:** the real user test with the lecturer and 5 students (protocol ready in `docs/USER_TEST.md`, Section 15).

2 Problem, users and goals

2.1 The problem

- Students do not know which topics they understand and in which order to study them.
- General AI chatbots answer from the internet, not from *this* course's slides, and they rarely say "I don't know".
- The lecturer cannot see which topics students ask about most or where they score lowest.

2.2 Users

User	What they need	Where in StudyMate
Student	Answers from the slides with the source, practice with hints, see weak topics, know what to study next	Ask, Practice, Progress, Studio, Exams, Study plan
Doctor (lecturer)	Upload lectures, see the most asked and weakest topics, follow students at risk, make exams	Content, Data dashboard, Weekly report, Exams, Announcements
Main admin / sub-admin	Manage doctors, students and permissions; evaluate and retrain the models	Admins, Doctors, Students, AI models, Activity log

2.3 Goals

1. Never answer from outside the slides; always give lecture and slide number.
2. Detect the topic of a question and link to practice on that topic.
3. Track each student's level per topic (red / amber / green) and suggest the next topic with a reason.
4. Give the lecturer a dashboard built from the Excel log.
5. Measure everything: accuracy, precision, recall, F1, confusion matrix, Top-1/Top-3, off-topic refusal, robustness.

2.4 User journey

1. Type a question → 2. Read 3 slides with lecture · slide → 3. Practise the same topic with hints → 4. Check topic colours → 5. Follow "Study next"

3 Requirements checklist

Every line of the project sheet (Project 1E and "Requirements for every project") and where it is met.

Requirement (project sheet)	What was delivered	Status
Flask app that runs with `python app.py`	`app.py` + `core/` package, 44 Jinja templates	✔ Done
Ask page: three closest passages with source, or "Not found"	`/ask` : quick answer + 3 slides with lecture · slide, slide image, highlighted words; Not-found gate	✔ Done
Practice page: question → hint → clearer hint → answer	`/practice` with two hints, the answer, review of mistakes, spaced repetition	✔ Done
Progress page: red / amber / green + "Study next"	`/progress` with the reason (prerequisites)	✔ Done
Admin page: upload lecture PDF and rebuild the index	Content page: upload one or many PDFs, rebuild, delete lectures/courses	✔ Done
Search TF-IDF + cosine (bonus BM25)	TF-IDF default; BM25, LSA, Hybrid, optional Neural	✔ Done
Topic classifier NB or LR	Both trained; best (macro-F1) is used	✔ Done
Simple mastery update after every answer	`core/mastery.py` (exponential update with hint penalty)	✔ Done
Lectures split into passages, about one per slide	200 passages from 7 PDFs (pypdf), OCR for picture slides	✔ Done
Question bank ≥ 5 questions × ≥ 8 topics	72 questions, 6 × 12 topics, each with hint 1, hint 2, answer, slide	✔ Done
≥ 300 student-style questions labelled by topic	466 questions (37–40 per topic)	✔ Done
Search test set of 30 questions with the correct slide	94 questions (the original 30 + 64)	✔ Done
Evaluation: accuracy + confusion matrix; Top-1/Top-3; 10 off-topic → Not found	Evaluation page + `scripts/evaluate.py` ; 10/10 original off-topic refused (47/50 extended)	✔ Done
Precision, recall, F1 per class; robustness tests	Per-class table, 2 confusion matrices, 5 robustness tests + Arabic	✔ Done
logs.xlsx: Questions, Answers, Visits — through one function `log_to_excel`	`core/logger.py` ; plus Feedback, Students, Exams sheets	✔ Done
Dashboard: most asked topics, lowest-score topics	Data dashboard with charts, students at risk, follow-ups, Excel download	✔ Done
Training data in CSV + one-page data card	`data/AI/*.csv` + `data/AI/data_card.md`	✔ Done
Delivered in one ZIP file	See Section 16	✔ Done
10-minute presentation (compare an answer with the original slide)	`docs/presentation_outline.md` ; the Ask page shows the slide image next to the answer	◐ Partly
Real user test with the lecturer	Protocol and results sheet in `docs/USER_TEST.md`	○ To do
Bonus: deployment	PythonAnywhere (live)	✔ Done
Bonus: Arabic questions	76-term glossary + Arabic interface; 10/10 Arabic test questions classified correctly	✔ Done

Requirement (project sheet)	What was delivered	Status
Bonus: language model that rephrases only from the retrieved passages	Extractive "Quick answer" built only from the 3 retrieved slides with their sources (no generative LLM, so nothing can be invented)	◐ Partly

4 System architecture

Browser — student, doctor, admin (English / Arabic, light / dark, phone / desktop)

Flask app (app.py)
Routes, sessions, roles & permissions, Jinja templates, i18n, background log writer

AI core (core/)
search.py · classifier.py · mastery.py · topics.py (study next) · autoeval.py · neural.py

Content pipeline
passages.py (pypdf + OCR) · autocourse.py (new courses) · slideview.py (slide images)

Data (CSV)
passages, topics, topic_map, question_bank, questions_labeled, search_test, offtopic_test, question_variants, glossary

Models (joblib)
search_index · topic_classifier (NB, LR, best) · search_threshold · split

Records
logs.xlsx (6 sheets) · JSON stores for accounts, chat, notifications, calendar, badges, audit

Offline tools
— scripts/build_passages.py · scripts/train.py · scripts/evaluate.py · 2 Google Colab notebooks · 92 pytest tests

4.1 What happens when a student asks a question

Question (EN / AR) → Arabic terms → English (glossary) → Normalise, fix spacing →
Topic classifier → topic + confidence → Not-found gate (similarity ≥ 0.02 and course-word coverage ≥ 0.8) →
Rank slides (TF-IDF) × topic boost → Top 3 + quick answer + slide image → log_to_excel("Questions")

4.2 Technology

Layer	Tools
Web	Python 3, Flask, Jinja2, sessions, HTML/CSS (own design system, RTL), vanilla JavaScript, Chart-style SVG charts
Machine learning	scikit-learn (TfidfVectorizer, cosine_similarity, MultinomialNB, LogisticRegression, TruncatedSVD, metrics), rank_bm25, joblib; optional fastembed
Documents	pypdf, pdfplumber, Pillow, optional Tesseract OCR, python-docx, openpyxl, pandas
Evaluation / notebooks	matplotlib, Google Colab, TensorFlow/Keras (MobileNetV2) for lecture images
Quality	pytest (92 tests), Playwright screenshots, Git/GitHub
Hosting	PythonAnywhere (WSGI), optional Render config

Code size: about 11,300 lines of Python, 44 templates, 1,400 lines of CSS/JS.

5 Data and data card

5.1 Sources and sizes (AI course)

File	Content	Size	Source
`lectures/*.pdf`	Lecture slides	7 PDFs, 260 slides	Lecturer (with permission)
`passages.csv`	One passage per slide (empty / title slides dropped)	200	pypdf extraction
`topics.csv`, `topic_map.csv`	12 topics, prerequisites, slide ranges	12 / 18 rows	Team, from the lecture agenda
`question_bank.csv`	Practice questions with hint 1, hint 2, answer, key ideas, slide	72 (6 × 12)	Team, from the slides
`questions_labeled.csv`	Student-style questions labelled with a topic	466	Team (draft with an AI assistant, checked against the slides)
`search_test.csv`	Questions with the slide(s) that answer them	94	Team, in words different from the slides
`offtopic_test.csv`	Questions not in the course	50	Team
`question_variants.csv`	Short student phrasings attached to a slide (index only)	94	Team
`arabic_glossary.csv`	Arabic → English course terms	76	Team

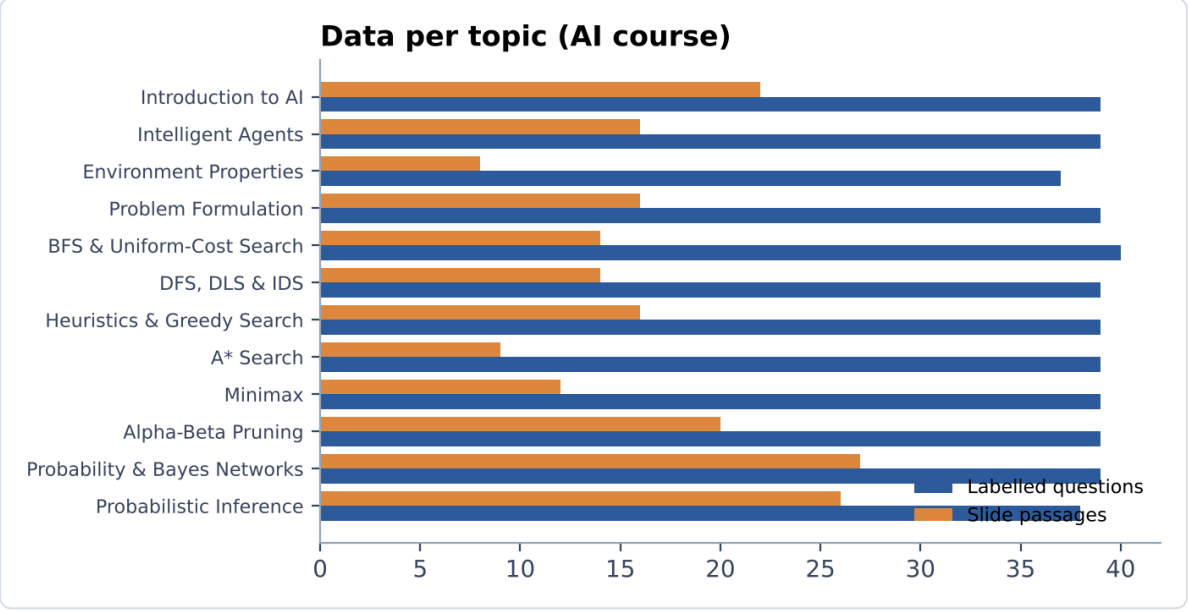


Figure 1. Labelled questions and slide passages per topic. Questions are balanced (37–40 per topic); passages follow the length of each lecture.

5.2 How the data was collected and cleaned

1. **Extraction:** every PDF page → one passage (pypdf). Repeated headers and footers ("LECTURE 3") are learned and removed; symbol-font letters (α, β, ε) become words; glued words are split ("Bayesnetwork" → "Bayes network"); slides with fewer than 6 words are dropped. Picture-only slides are read with OCR when Tesseract is available, or with text imported from the Colab notebook.
2. **Topics:** 12 topics from the lecture agenda; each slide range is mapped to a topic in `topic_map.csv`; prerequisites (e.g. A* needs Heuristics) are stored in `topics.csv`.
3. **Labelling rule:** a question takes the topic of the slide that answers it; if two topics fit, the more specific one wins ("is h admissible for A*?" → A* Search).
4. **Realistic wording:** informal language, very short questions and real spelling mistakes ("breath first search", "huristic", "alfa beta").
5. **No leakage:** the search test, the off-topic set and the variants never overlap with the training questions (checked by an automatic test).

5.3 Privacy

- The slides contain no personal data; lecturers' names on title slides are not indexed.
- Students register with name, e-mail and a numeric university code; passwords are stored only as salted hashes. The Excel log uses the student code, not the name.
- Account files (`data/students.json`, `doctors.json`, chat, notifications...) are excluded from Git by `.gitignore`.

6 The AI models

6.1 Search: finding the slide

Each slide is represented by its text, its topic name and an **index-only expansion**: the question-bank questions and answers written for that slide and the short student phrasings from `question_variants.csv`. The student never sees this extra text; it only helps student wording reach the right slide.

- **TF-IDF + cosine (default):** unigrams and bigrams, sub-linear term frequency, English stop words + a course stop list (words like "explain", "lecture", "difference" that should not decide a match), `max_df = 0.6`.
- **BM25 (bonus):** Okapi BM25 over the same tokens, rescaled to the cosine range.
- **LSA:** 100-dimension truncated SVD of the TF-IDF matrix, so related words can match.
- **Hybrid:** 0.5 × TF-IDF + 0.5 × LSA.
- **Neural (optional):** `bge-small-en-v1.5` sentence embeddings via fastembed (CPU, free); hidden automatically when the model cannot be downloaded (as on free PythonAnywhere).
- **Topic re-ranking:** score × (1 + w · P(topic of the slide | question)), using the classifier's probabilities.

6.2 The "Not found" gate

A question is answered only if **both** rules pass: (1) the best TF-IDF similarity is at least **0.02**, and (2) at least **0.8** of the question's content words exist in the course vocabulary (close spellings allowed). The two values are found by a grid search that first maximises refused off-topic questions and then answered course questions. The grid search uses only the *training* student questions and the even rows of the off-topic file; the odd rows are kept for an honest report. The main admin can also change the threshold from the Content page.

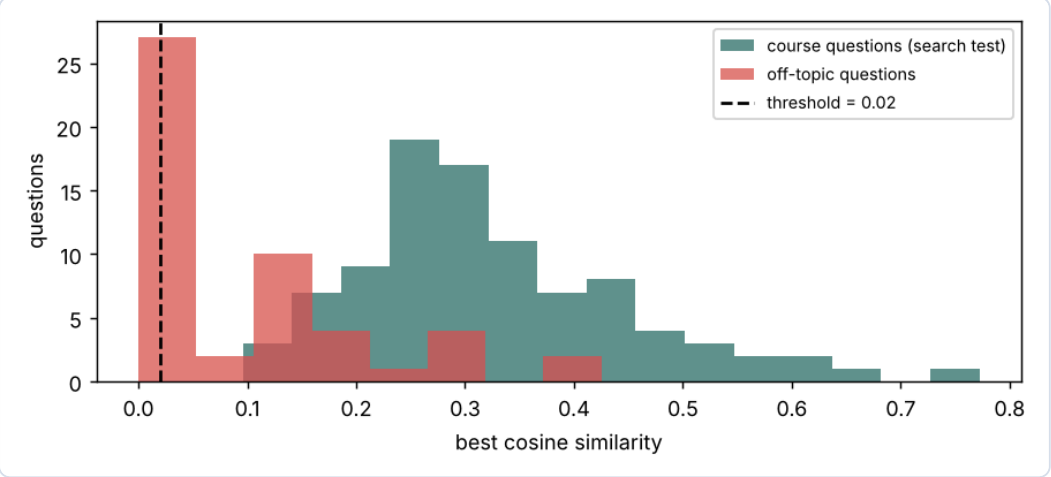


Figure 2. Best similarity for course questions (green) and off-topic questions (red), with the chosen threshold.

6.3 Topic classifier

A **FeatureUnion** of word TF-IDF (1–2-grams) and character TF-IDF (3–5-grams inside words — this survives typos), followed by either **Multinomial Naive Bayes** (α = 0.1) or **Logistic Regression** (C = 10). The split is stratified 80/20 (seed 42). The training part is enlarged with the 72 question-bank questions

and the 200 slide texts, each labelled with its topic (never used for testing). The model with the best macro-F1 is saved as the default.

6.4 Mastery and "Study next"

```
mastery_new = mastery_old + 0.35 × (outcome − mastery_old)
outcome = 1.0 (correct, no hint) · 0.7 (one hint) · 0.4 (two hints) · 0.0 (wrong)
colour  = red < 0.40 ≤ amber < 0.70 ≤ green
```

Study next: among topics that are not green, prefer one whose prerequisites are already green ("ready"); if a weak topic is needed by later topics, the reason says so (e.g. "Heuristics is needed before A*"). Course order breaks ties. Weak topics come back after 1, 3 and 7 days (spaced repetition).

6.5 Other intelligent features

- **Automatic evaluation for any uploaded course** (`core/autoeval.py`): one sentence is held out from each slide and used as a question, so a new course gets Top-1/Top-3 numbers without a hand-made test set.
- **New courses without labels:** topics are created from the lecture structure; the topic of the closest slides replaces the classifier.
- **Exam grading:** essay answers are scored by similarity to the model answer.
- **Students at risk:** inactivity + low level → notification to the course doctor.

7 Evaluation and results

All numbers come from `scripts/evaluate.py` (saved to `static/reports/metrics.json`) and are shown on the Evaluation page of the site.

7.1 Topic classifier

Model	Accuracy	Macro-F1	5-fold CV accuracy
Logistic Regression (used)	**91.5%**	**0.913**	91.2% ± 2.1
Multinomial Naive Bayes	84.0%	0.835	91.2% ± 2.6

Test set: 94 held-out questions (stratified). Both models reach the same cross-validated accuracy; on this particular split Logistic Regression is better, so it is the default.

Topic	Precision	Recall	F1	n
Introduction to AI	0.80	1.00	**0.89**	8
Intelligent Agents	1.00	0.75	**0.86**	8
Environment Properties	0.78	1.00	**0.88**	7
Problem Formulation	1.00	0.88	**0.93**	8
BFS & Uniform-Cost Search	0.89	1.00	**0.94**	8
DFS, DLS & IDS	1.00	0.88	**0.93**	8
Heuristics & Greedy Search	0.83	0.62	**0.71**	8
A* Search	0.89	1.00	**0.94**	8
Minimax	0.89	1.00	**0.94**	8
Alpha-Beta Pruning	1.00	0.88	**0.93**	8
Probability & Bayes Networks	1.00	1.00	**1.00**	8
Probabilistic Inference	1.00	1.00	**1.00**	7
Macro average	0.92	0.92	**0.91**	94

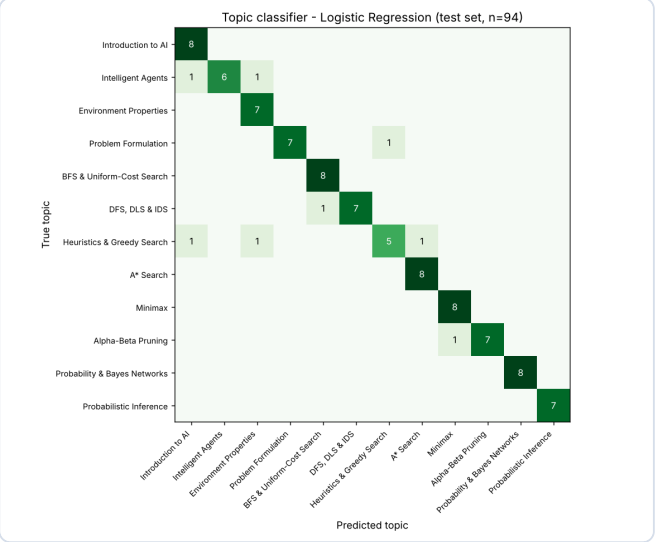


Figure 3. Confusion matrix, Logistic Regression (test set, n = 94).

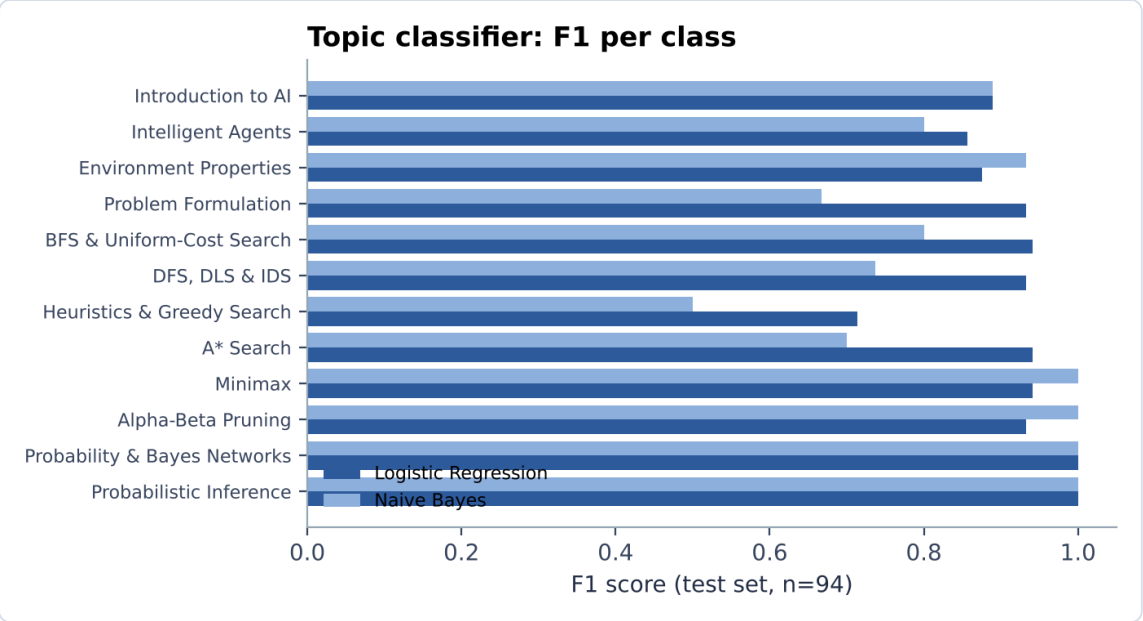


Figure 4. F1 per topic for both models. The weakest class is Heuristics & Greedy Search, which shares words with A* and Problem Formulation.

Error analysis (all 8 errors of Logistic Regression)

Question (test set)	True topic	Predicted	Why it happened
uninformed vs informed search	Problem Formulation	Heuristics & Greedy Search	Asks about two topics at once; "informed" pulls it to heuristics.
why use ids instead of bfs	DFS, DLS & IDS	BFS & Uniform-Cost Search	Compares IDS with BFS, so both topic words appear.
what is an intelligent agnet	Intelligent Agents	Introduction to AI	Typo "agnet" hides the key word; char n-grams were not enough.
what does h(n) mean	Heuristics & Greedy Search	Environment Properties	Very short, only a symbol – almost no words to learn from.
does pruning change the result of minimax	Alpha-Beta Pruning	Minimax	"pruning" and "minimax" both present; boundary question.
where is the intelligence in heuristics	Heuristics & Greedy Search	Introduction to AI	"intelligence" is a strong Introduction-to-AI word.
evaluation function f(n)	Heuristics & Greedy Search	A* Search	f(n) is taught with A*; the label follows the lecture order.
how do i specify a task environment	Intelligent Agents	Environment Properties	"task environment" sits on the border of two lectures.

7.2 Search

94 hand-written questions, each with the slide(s) that really answer it. A hit at Top-1 means that slide is ranked first; Top-3 means it is among the three shown.

Method	Top-1	Top-3	Notes
TF-IDF + cosine (default)	**80.9%**	87.2%	word + bigram TF-IDF, sub-linear tf, course stop list
BM25 (bonus)	79.8%	**90.4%**	rank_bm25 Okapi; best Top-3
Hybrid TF-IDF + LSA	79.8%	89.4%	0.5 × exact words + 0.5 × latent semantic
Semantic LSA	74.5%	89.4%	100-d SVD of the TF-IDF matrix
Neural (optional)	Colab	Colab	bge-small sentence embeddings; measured in the notebook

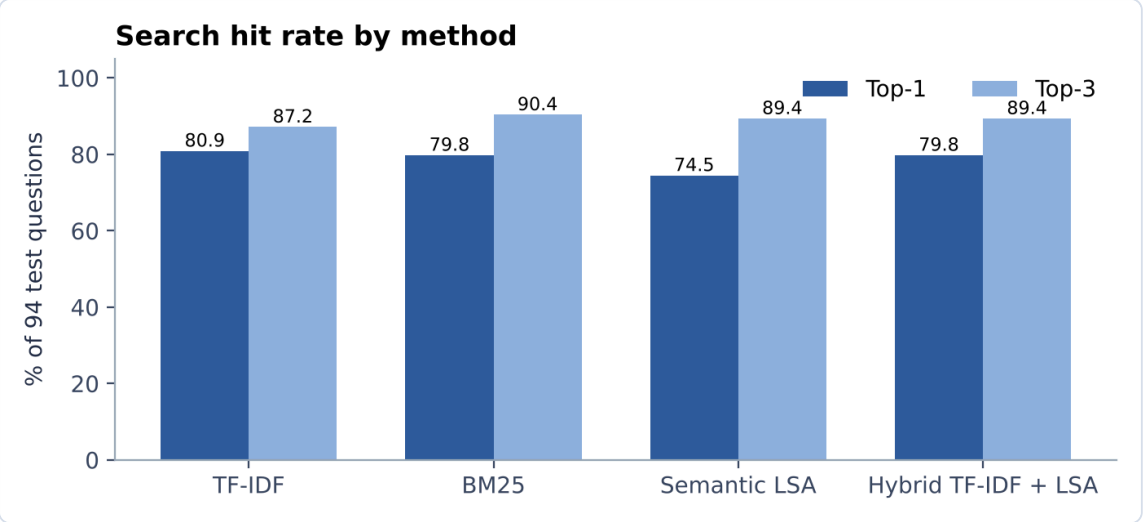


Figure 5. Top-1 and Top-3 of the four search methods on the 94 test questions.

> **Reading the numbers.** TF-IDF is the best at putting the right slide first; BM25 is the best at getting it into the three cards the student sees. TF-IDF stays the default because the "Not found" gate is tuned on TF-IDF scores. Remaining misses are mostly paraphrases with no shared word ("fool a human judge in an imitation game" → Turing test) and slides that are only pictures.

7.3 Off-topic questions ("Not found")

Set	Refused	Note
The 10 questions required by the project sheet	**10 / 10**	World cup, koshari, capital of Australia, SVM, k-means, derivative of sin x...
Extended set (all 50)	47 / 50	Missed: speed of light, AC vs DC current, CNN image recognition
Held-out half (never used for tuning)	22 / 25	The honest number
Course questions still answered	91 / 94 search tests · 92.5% of student questions	The price of refusing off-topic questions

The three misses share words with the course (*light*, *difference*, *network*, *images*), which shows the limit of a word-based gate.

7.4 Robustness

Test	Example	Classifier accuracy	Search Top-3*
original	why is the real world continuous	91.5%	86.2%
typos	why is the real world cotnuous	89.4%	61.7%
UPPER CASE	WHY IS THE REAL WORLD CONTINUOUS	91.5%	86.2%
very short (2 key words)	real world	86.2%	60.6%
extra words	hi doctor, quick question please: why is the real world cont	89.4%	86.2%
Arabic questions (bonus)	ايه هو البحث بالعرض	100.0%	100% answered

* Search Top-3 here is measured *with* the Not-found gate active, so a refused question counts as a miss.

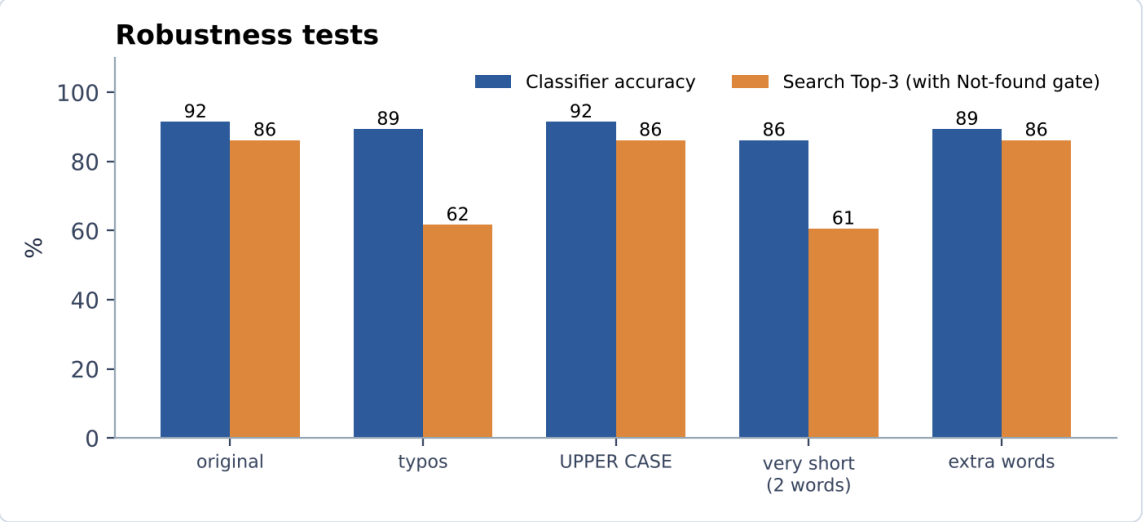


Figure 6. Classifier and search under typos, upper case, very short questions and extra words.

7.5 Second course and automatic evaluation

The same pipeline runs on any uploaded course. For a course without a hand-made test, the automatic hold-out test gives a lower bound (the question is a sentence removed from the slide, so the slide has lost its best words):

Course	Slides	Hold-out queries	TF-IDF Top-1 / Top-3	BM25 Top-1 / Top-3	Other-course questions refused
Artificial Intelligence	200	30	26.7% / 46.7%	26.7% / 46.7%	30 / 40
Automotive Engineering (MAM 336)	108	82	17.1% / 30.5%	19.5% / 36.6%	37 / 40

These are much stricter than the hand-written test and are used to compare methods and to warn the lecturer when slides are too short or picture-only.

8 Improvement round (12 November)

After the first full evaluation, the weak points were addressed in order of value:

1. **Bigger test sets** — with 30 questions one question moves the score by 3.3 points. The search test grew to 94 questions and the off-topic set to 50.
2. **Query-variant expansion** — 94 short student phrasings attached to their slides (index only).
3. **More labelled questions** — 360 → 466, focused on topics that are easily confused.
4. **Leak-free threshold** — the threshold used to be tuned on the same questions it was reported on; now tuning and reporting use different halves.
5. **Neural method in Colab** — measured on the same 94 questions in the notebook, with a clear rule: use it as the default only if Top-1 > 80% and better than TF-IDF.

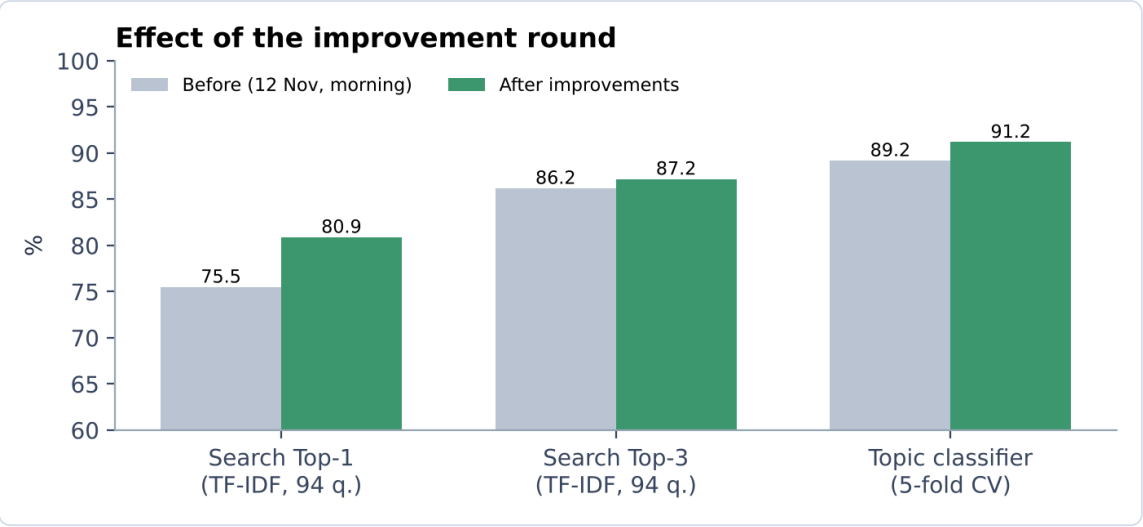


Figure 7. Same 94 test questions before and after the improvement round. Search Top-1 +5.4 points; classifier +2.0 points.

Measure	Before	After	Change
Search test size	30	94	× 3.1
Off-topic test size	10	50	× 5
Labelled questions	360	466	+106
TF-IDF Top-1 (same 94 questions)	75.5%	**80.9%**	+5.4
TF-IDF Top-3	86.2%	87.2%	+1.0
Classifier, 5-fold CV	89.2%	**91.2%**	+2.0

9 The application — student side

9.1 Sign in and registration

One sign-in page for everybody (username, e-mail or university code). Registration first asks "student or doctor". Students must give a numeric university code; doctors give their department and wait for the main admin's approval. Hints under the username and password appear only when the typed value breaks the rule; a button can generate a strong password.

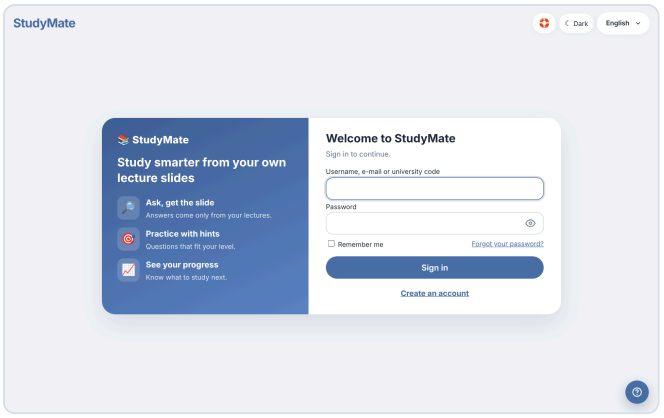


Figure 19. Sign-in page.

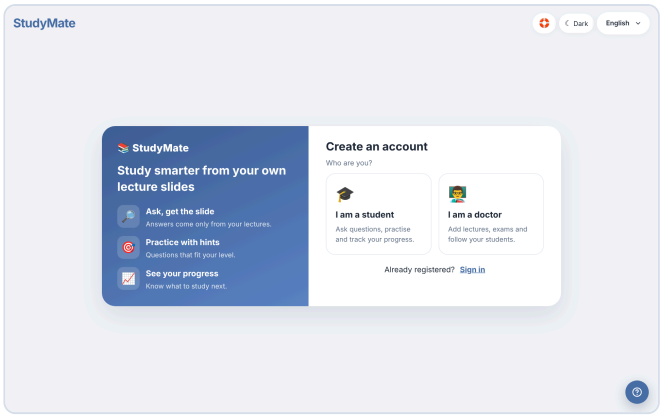


Figure 20. Step 1 of registration: student or doctor.

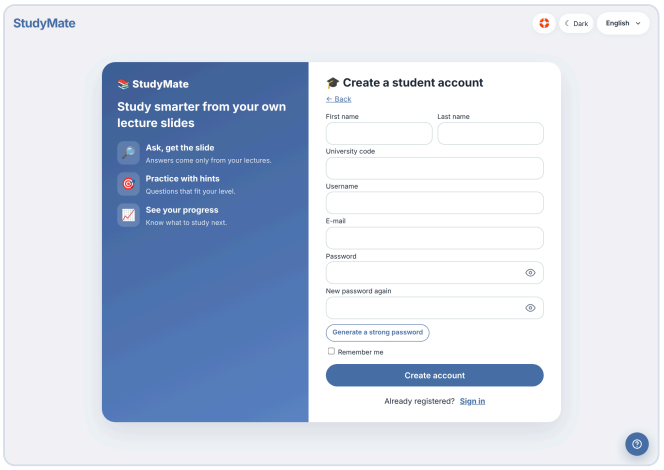


Figure 21. Student registration with the required university code.

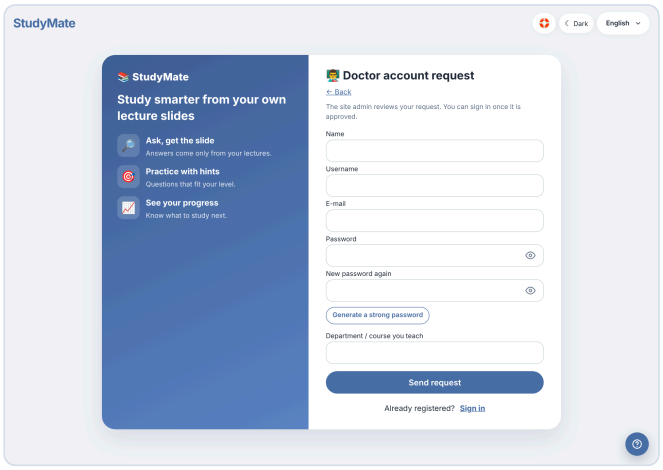


Figure 22. Doctor request with the required department.

9.2 Home

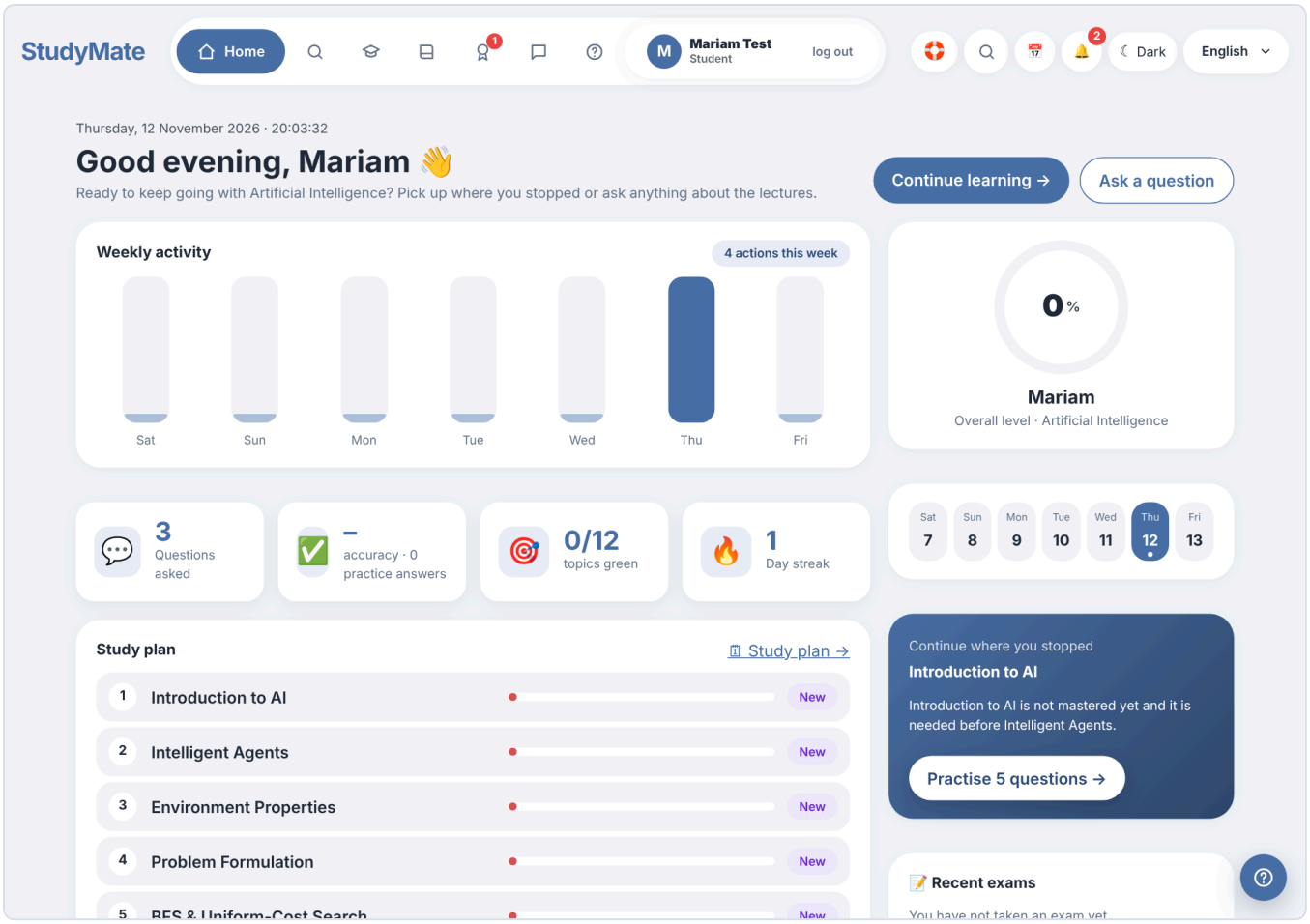


Figure 8. Student home: greeting, weekly activity, level, study plan and recent exams.

9.3 Ask — the core page

The detected topic and its confidence, a quick answer built only from the retrieved slides (each sentence with its lecture · slide tag), then the three slides with the slide image, the highlighted matching words and the similarity score. The side panel shows the student's topics with coloured dots and the "Study next" box.

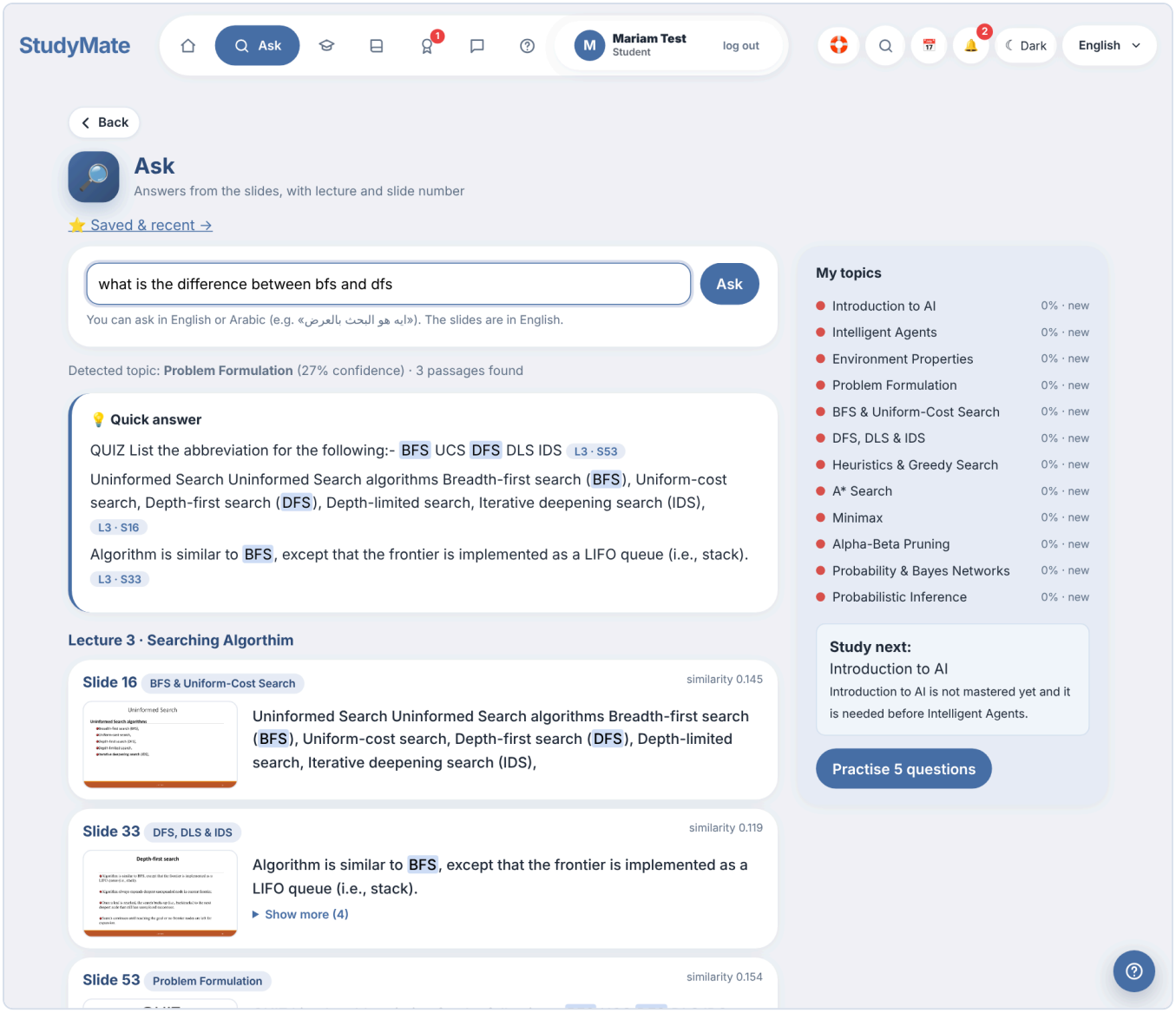


Figure 9. Ask: "what is the difference between bfs and dfs" → Lecture 3 slides with source, image and quick answer.

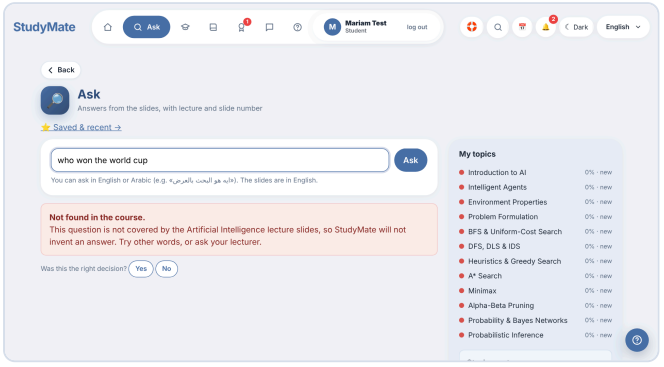


Figure 23. An off-topic question is refused: "Not found in the course".

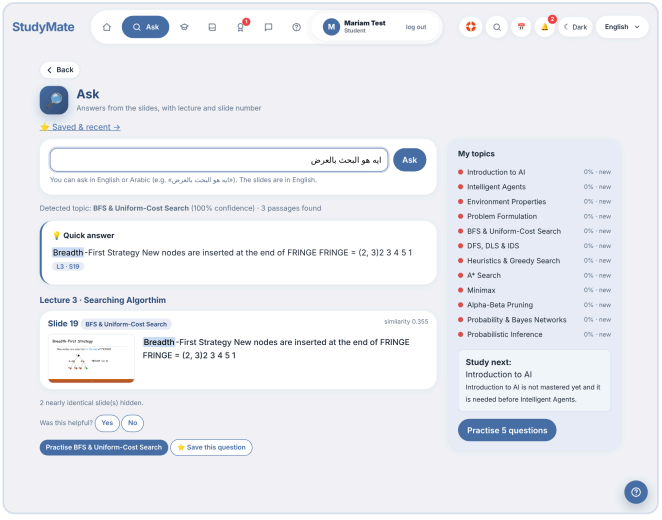


Figure 24. An Arabic question («ايه هو البحث بالعرض») is answered from the English slides.

9.4 Practice and progress

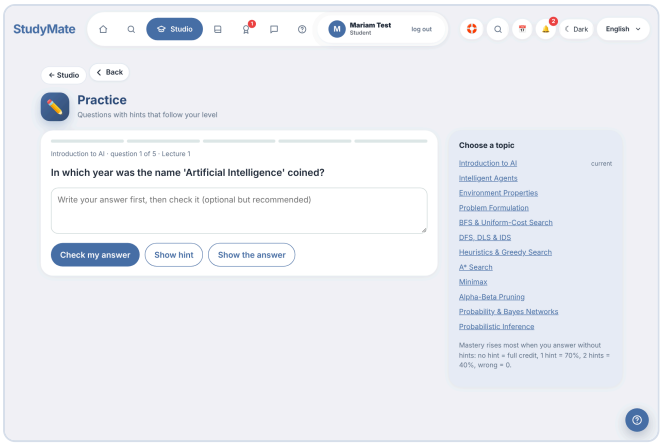


Figure 25. Practice: question, hint, clearer hint, then the answer.

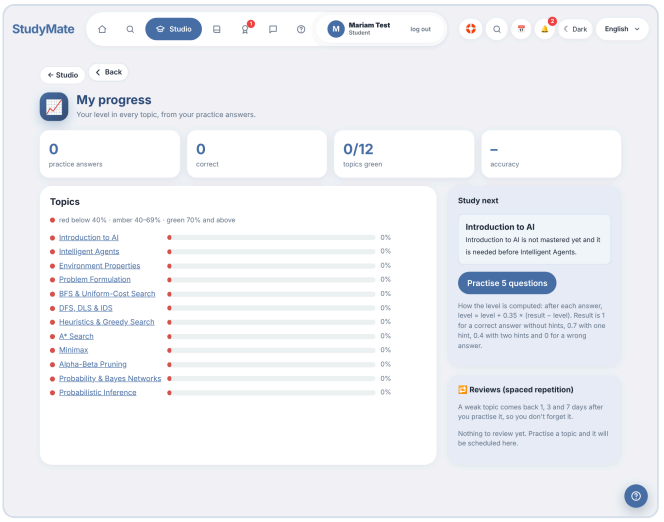


Figure 26. Progress: topics in red / amber / green, "Study next" with its reason, reviews due.

9.5 Study studio, exams and study plan

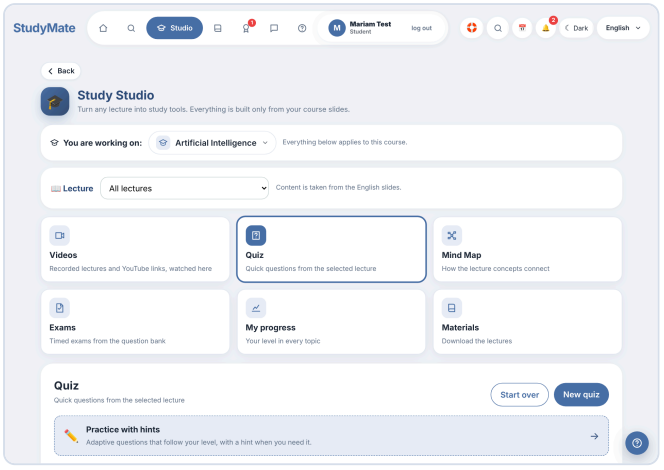


Figure 27. Studio: videos, quiz, mind map, exams, materials for the chosen course.

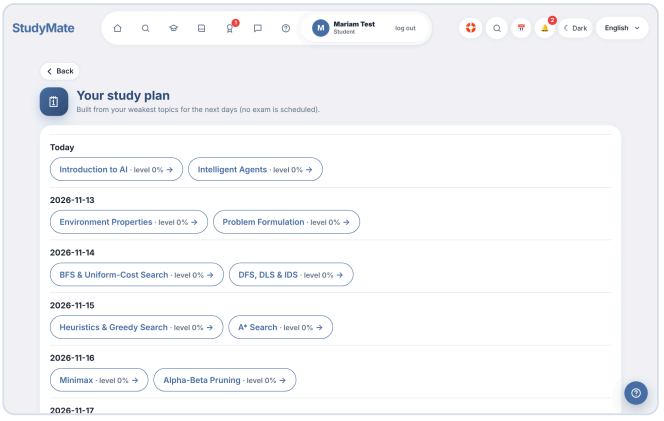


Figure 28. Study plan: weak topics spread over the coming days.

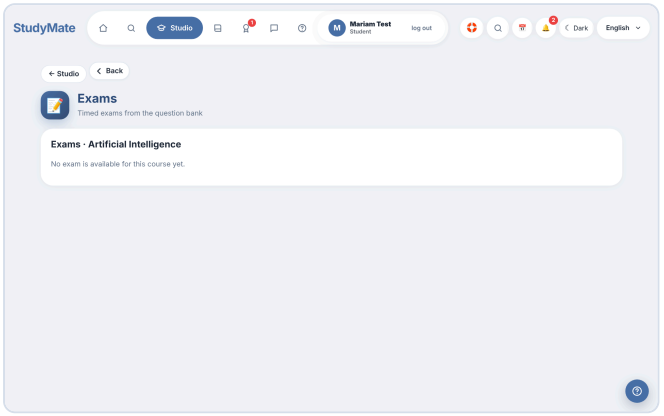


Figure 29. Exams built from the lecturer's question bank.

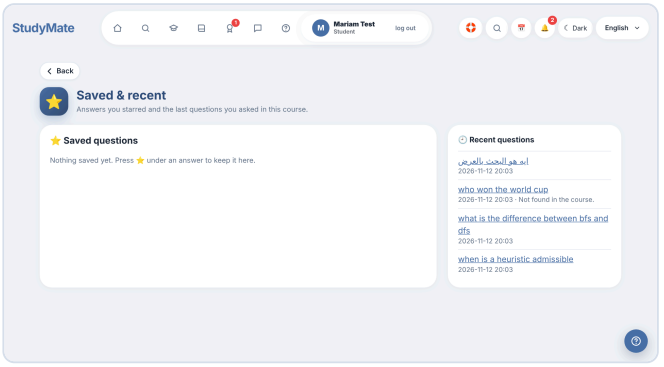


Figure 30. Saved answers and recent questions.

9.6 Communication and motivation

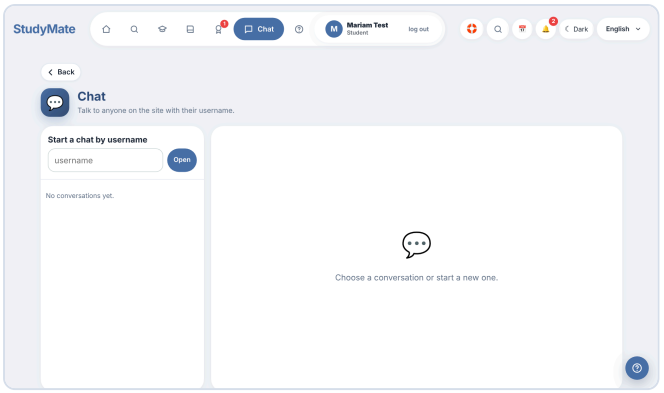


Figure 31. Chat: a chat starts only after the other side accepts the request; blocking is available.

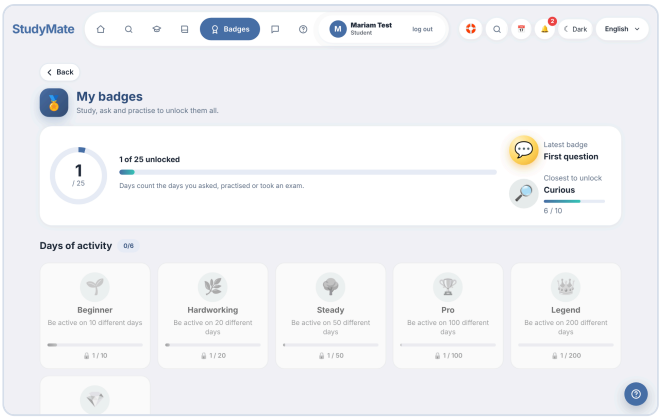


Figure 32. Badges for activity and progress.

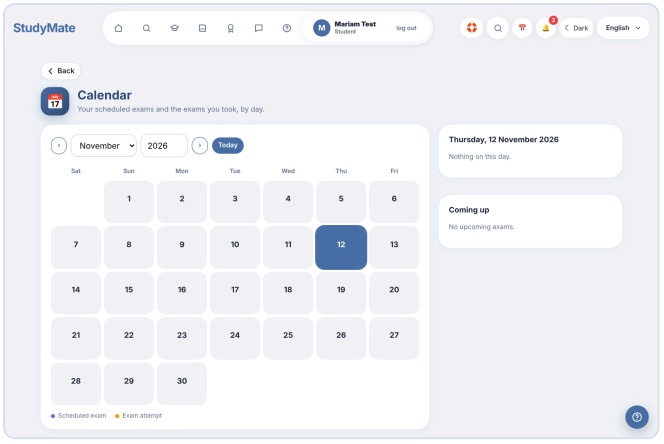


Figure 33. Calendar with scheduled exams.

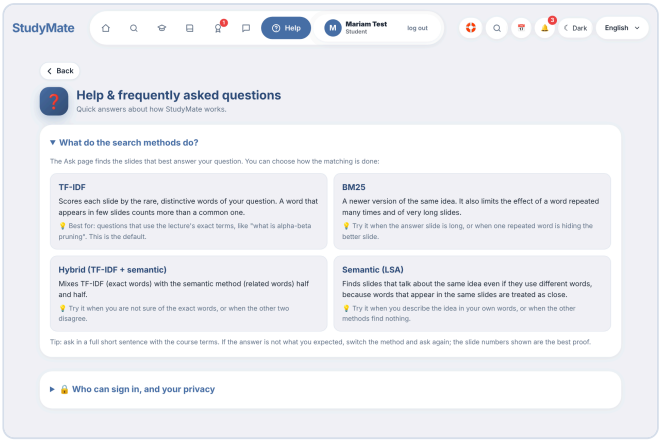


Figure 34. Help page: how the search methods work and the privacy note.

9.7 Mobile and dark mode

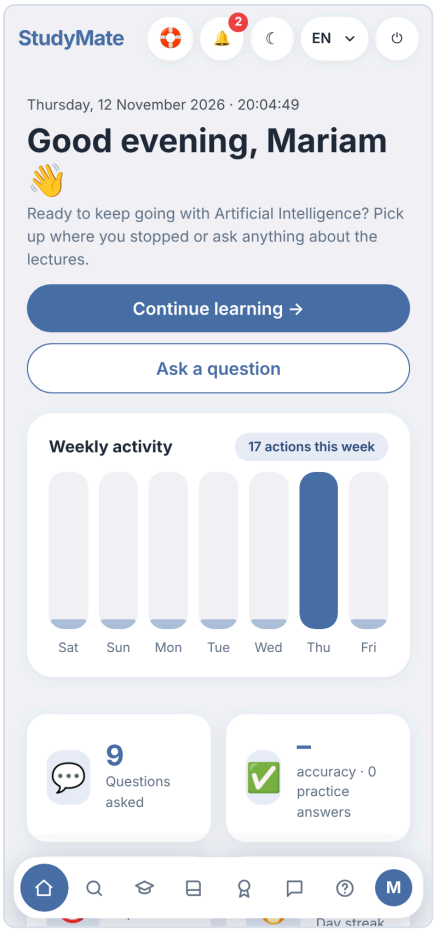


Figure 10. Phone: home.

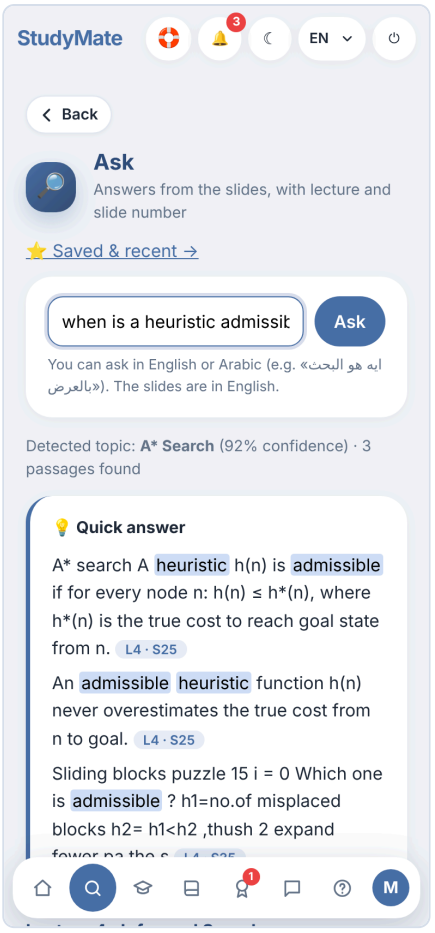


Figure 11. Phone: Ask.

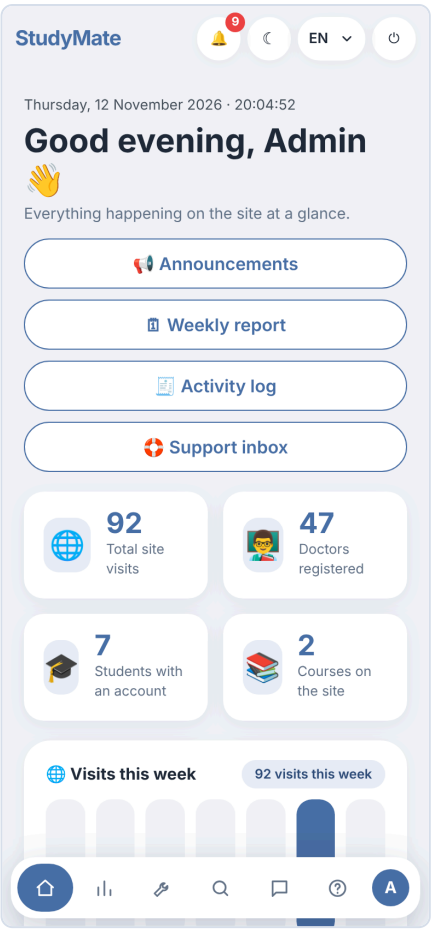


Figure 12. Phone: admin home.

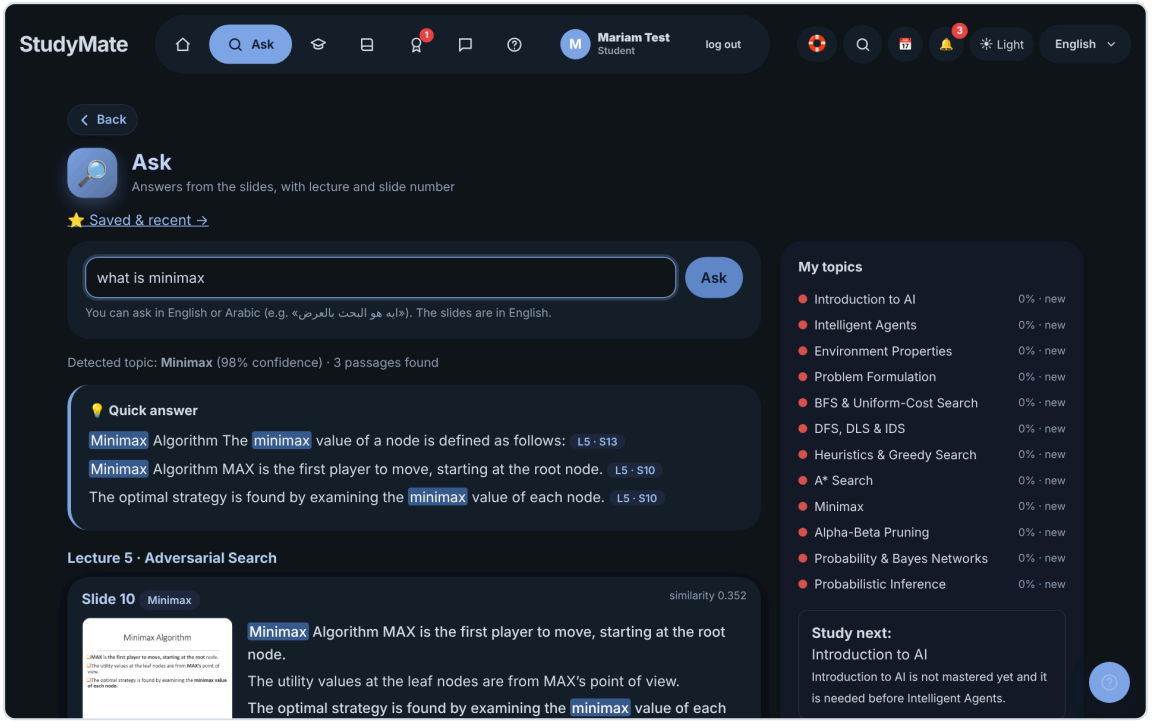


Figure 13. Dark mode (remembered per browser).

10 The application — lecturer and admin side

Three staff roles: the **main admin** (owner of the site), **sub-admins** with selected permissions (content, exams, badges, data, students, chat, support, doctors, announcements) and **doctors**, who see only the courses registered under their name and their students.

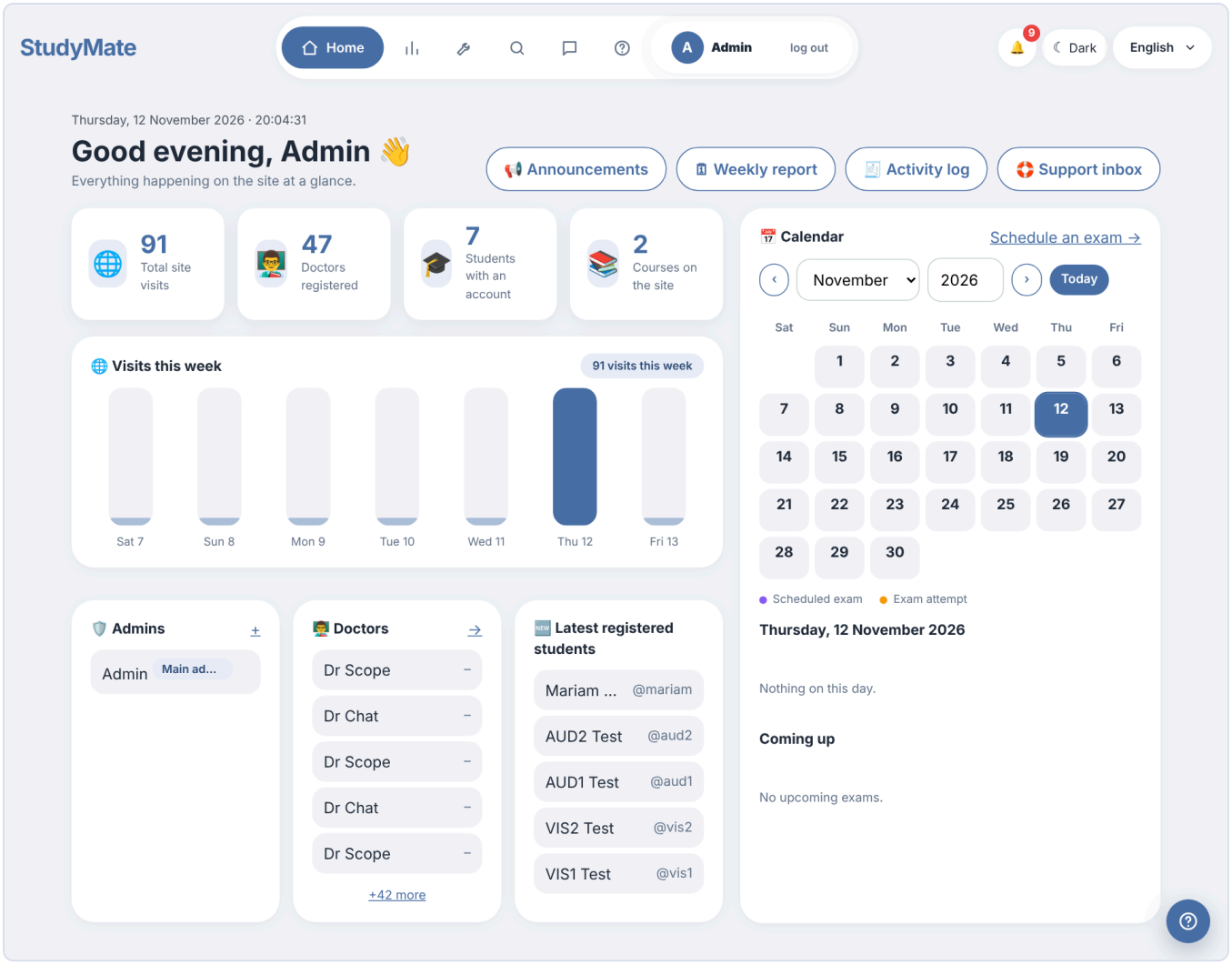


Figure 14. Admin home: site numbers, weekly visits, calendar, doctors, latest students.

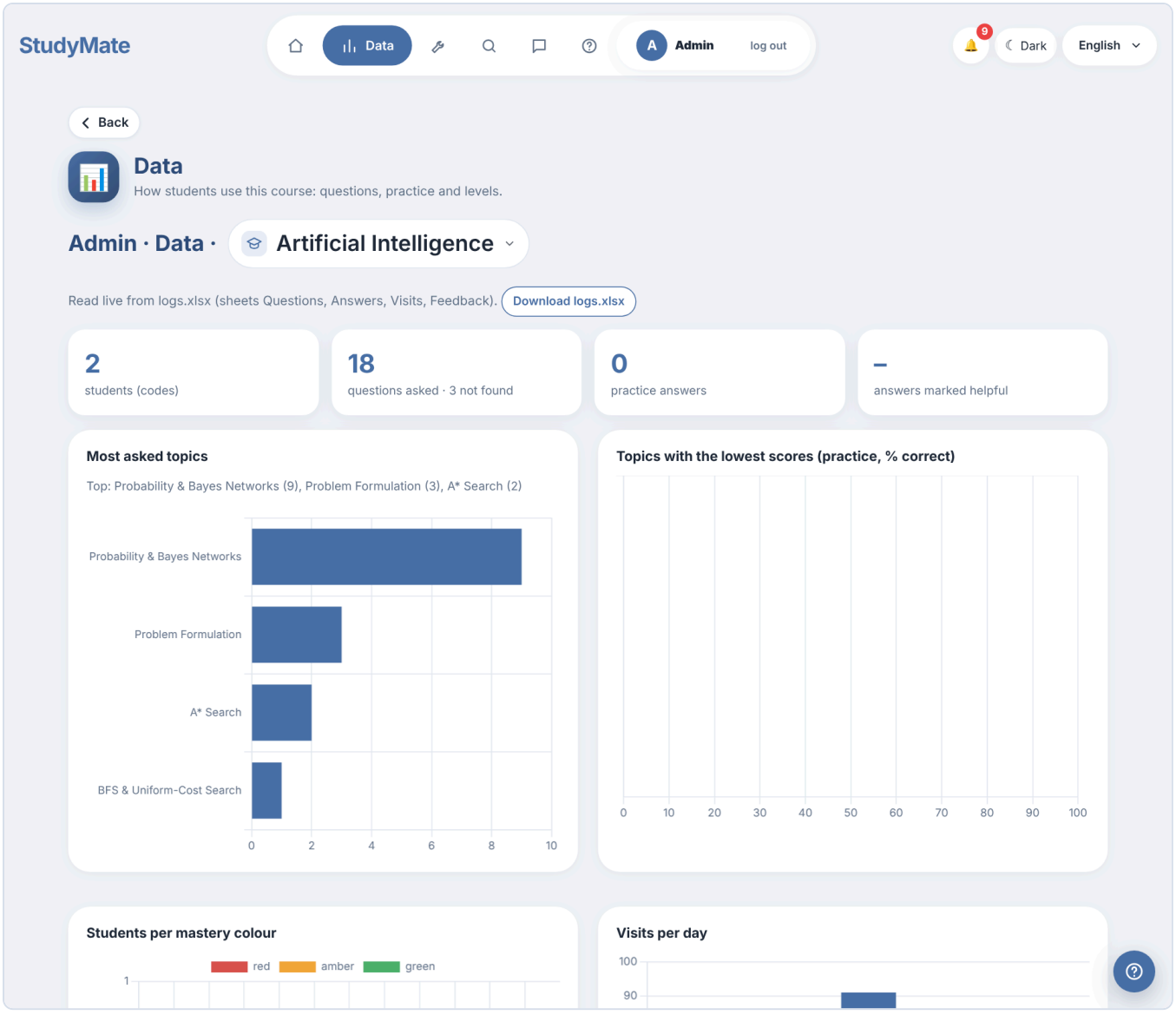


Figure 15. Data dashboard (required): most asked topics, topics with the lowest scores, students per mastery colour, visits per day; read from logs.xlsx.

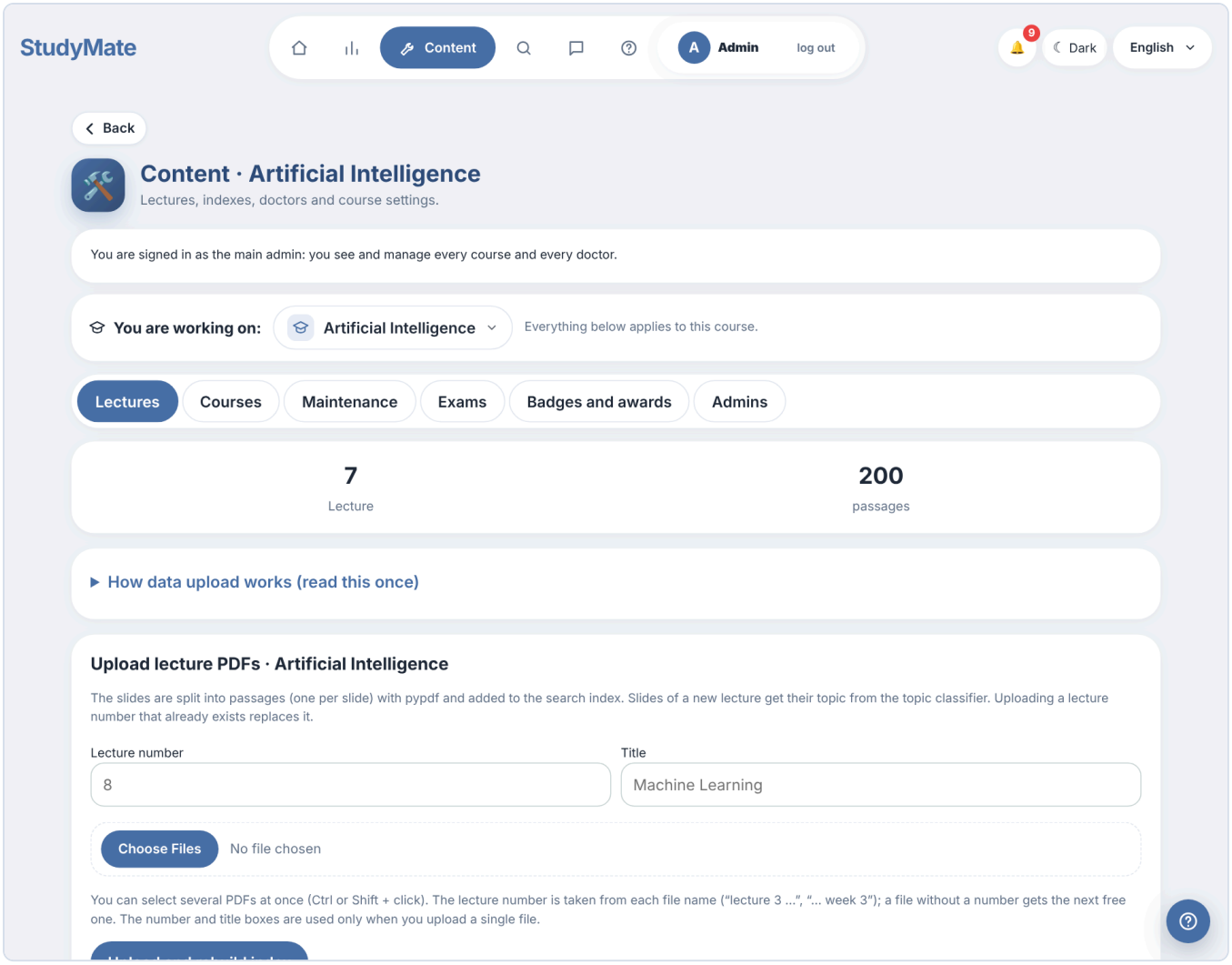


Figure 16. Content: upload lecture PDFs, rebuild the index, maintenance, exams, badges.

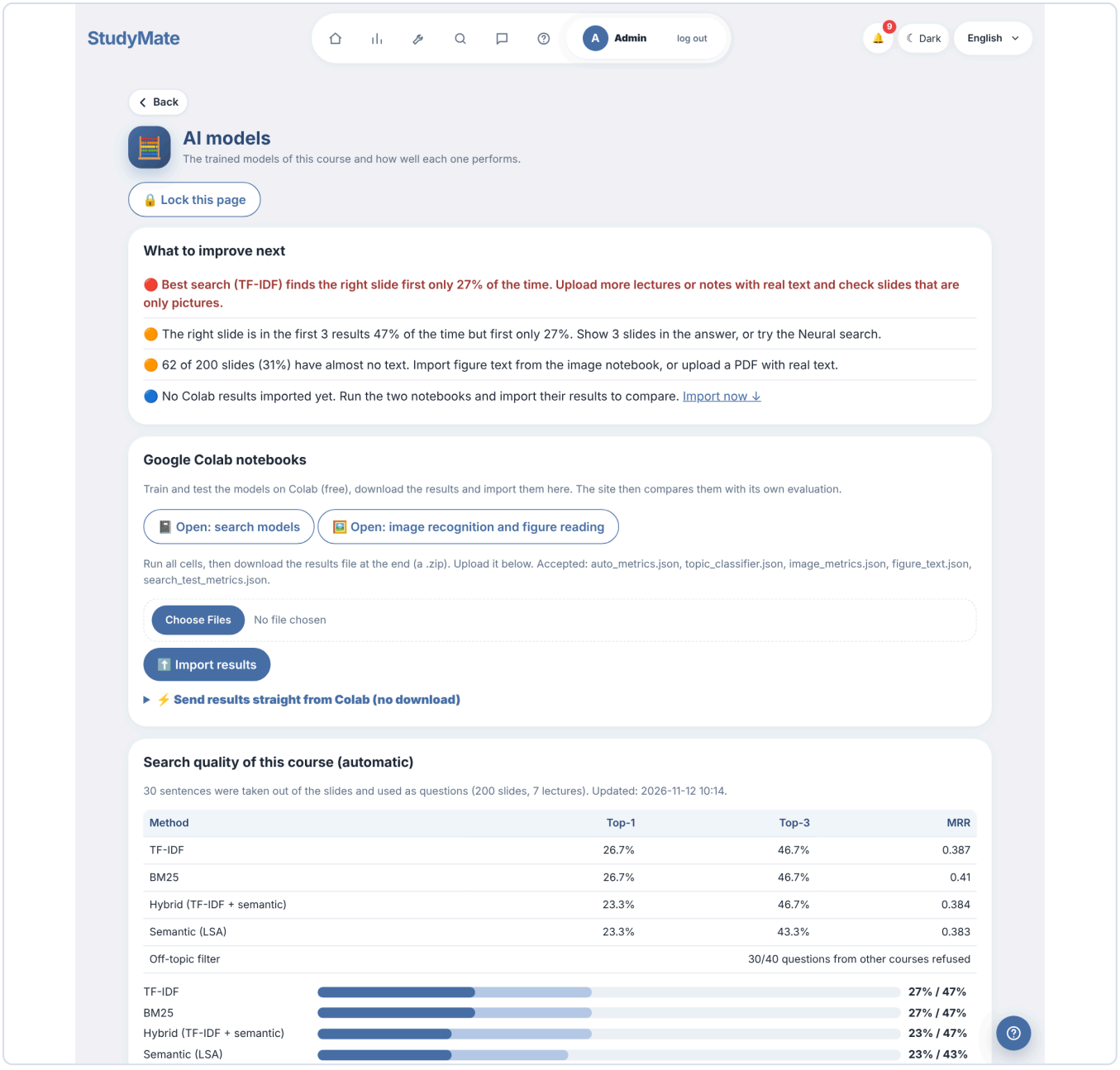


Figure 17. AI models page (main admin, protected by a second password): advice, Colab import, automatic evaluation, classifier metrics and confusion matrices.

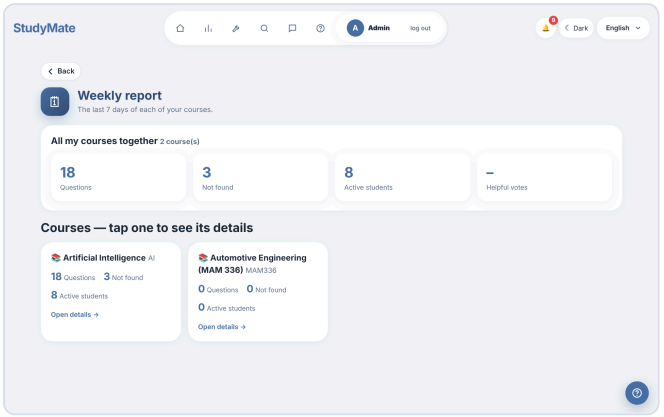


Figure 35. Weekly report: totals of the user's courses and clickable course tiles.

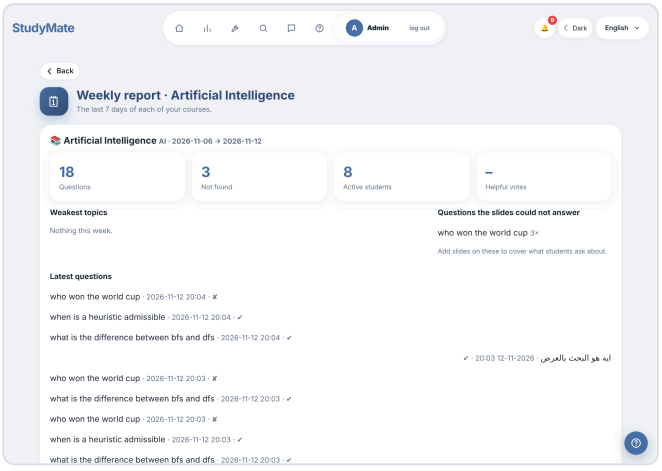


Figure 36. Weekly report of one course: weakest topics, questions the slides could not answer, latest questions.

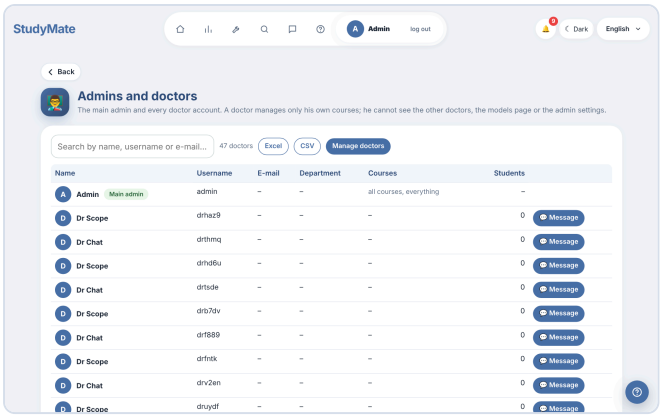


Figure 37. Admins and doctors, with their department.

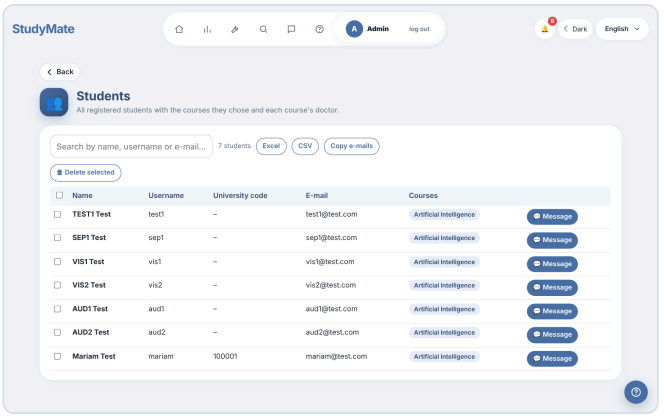


Figure 38. Students with university code; bulk delete and export (Excel / CSV).

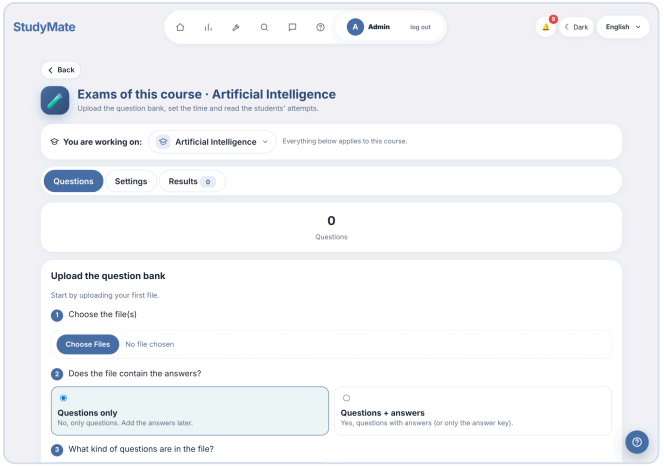


Figure 39. Exam management: upload a question bank, settings, results.

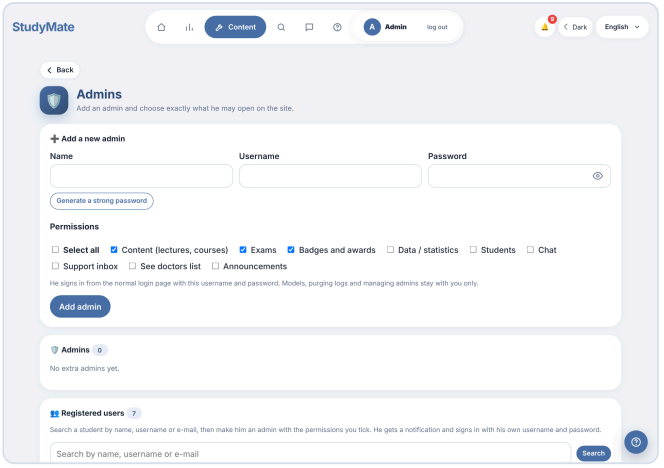


Figure 40. Adding a sub-admin with selected permissions.

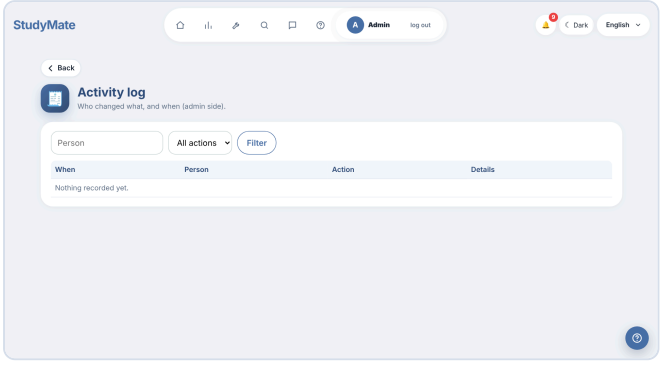


Figure 41. Activity log: who did what and when.

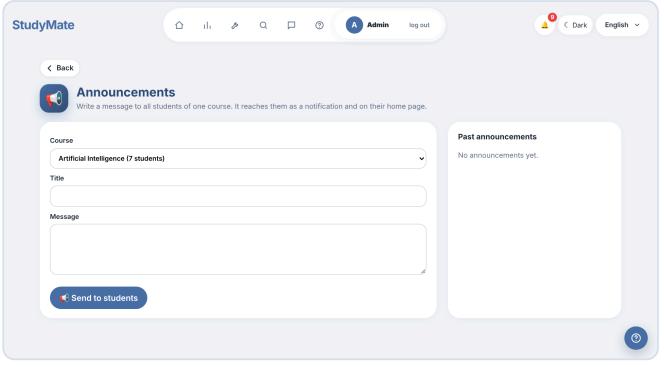


Figure 42. Announcements to the students of a course.

11 Excel database and dashboard

All records are written through one function, `log_to_excel(sheet, record)` in `core/logger.py` (page visits use a background queue so a page never waits for Excel). Missing sheets or columns are added automatically.

Sheet	Columns
Questions	id, time, student_code, question, topic, confidence, first_slide, found, method, helpful, course
Answers	id, time, student_code, question_id, topic, correct, hints_used, mastery_after, course
Visits	id, time, student_code, page, course
Feedback	id, time, student_code, question_log_id, helpful, comment, course
Students	id, time, student_code, first_name, last_name, email
Exams	id, time, student_code, attempt_id, n_questions, correct, score_pct, duration_sec, lang, wrong, course

The lecturer's dashboard reads these sheets and shows the most asked topics, the topics with the lowest practice scores, students per mastery colour, visits per day, students at risk (with a reminder button) and a download of `logs.xlsx`. The weekly report summarises the last 7 days per course and notifies the course owner once a week.

12 Google Colab notebooks

Colab is used as a free laboratory (GPU, internet for model downloads); the site stays on PythonAnywhere. Results come back to the site in two ways: download the results `.zip` and import it on the AI models page, or paste the site address and a secret token into the last cell so the notebook sends the results directly.

Notebook	What it does	Output files
`StudyMate_Search_Models.ipynb`	Data overview; hold-out evaluation of 4 methods; comparison across courses; one question through all methods; **new: hand-written test incl. the neural model**; topic-classifier cross-validation	auto_metrics.json, search_test_metrics.json, topic_classifier.json, charts
`StudyMate_Image_Recognition.ipynb`	Extracts the pictures from the lecture PDFs, removes logos (average hash), trains MobileNetV2 (transfer learning) to tell which lecture a picture comes from, builds an image-to-slide search, reads text inside pictures (OCR)	image_metrics.json, figure_text.json, image_model.keras, charts

Image notebook — honest note. The stored run was made on a CPU machine without internet, so a small CNN was trained from scratch on 250 pictures: 26% accuracy over 7 lectures (chance ≈ 14%), and finding the right lecture of a picture 38% of the time. On Colab with ImageNet weights this is expected to rise; run the notebook and import the results.

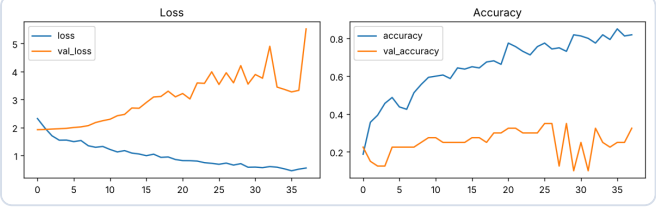


Figure 43. Image CNN: training curves (CPU run).

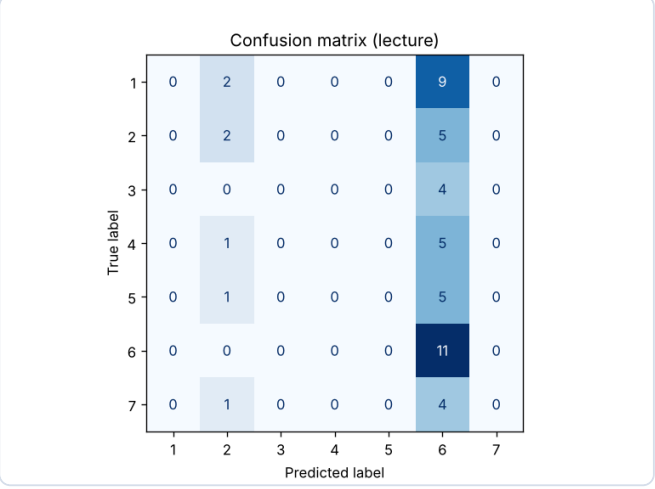


Figure 44. Image CNN: confusion matrix by lecture (CPU run).

13 Security and privacy

- Passwords are stored only as salted hashes (Werkzeug); rules: 8–15 characters with capital, small, digit and symbol for new accounts.
- Sign-in lock: 5 wrong attempts → 15-minute pause for that username.
- Roles and per-permission access (`perm_required`); a doctor can open only his own courses (`can_manage`).
- The AI models page needs a second password; the Colab upload needs a secret token that is not stored in Git.
- Doctor accounts are created only after the main admin approves the request; every admin action is written to the activity log.
- Chat: no message before the other side accepts; blocking for every user type; closing a chat deletes it from that side.
- Personal data files are excluded from the repository; the Excel log uses student codes.

14 Testing, deployment and development history

14.1 Automatic tests

92 pytest tests run the whole application with a test client (about 3 minutes): data requirements of the sheet (sizes, no overlap between training and test sets), search and Not-found behaviour, classifier, mastery, every page for every role, registration rules (university code, department), permissions, chat requests and blocking, exams, notifications, the weekly report and the Colab upload. Tests use temporary files so they never touch real data.

```
$ python -m pytest -q
92 passed in 168.75s
```

14.2 Deployment

PythonAnywhere (free plan), Python 3.10, WSGI file with `SECRET_KEY` and `LECTURER_PASSWORD`. Updating the live site:

```
cd ~/StudyMate && git pull        # then Web tab → Reload
```

14.3 Development history

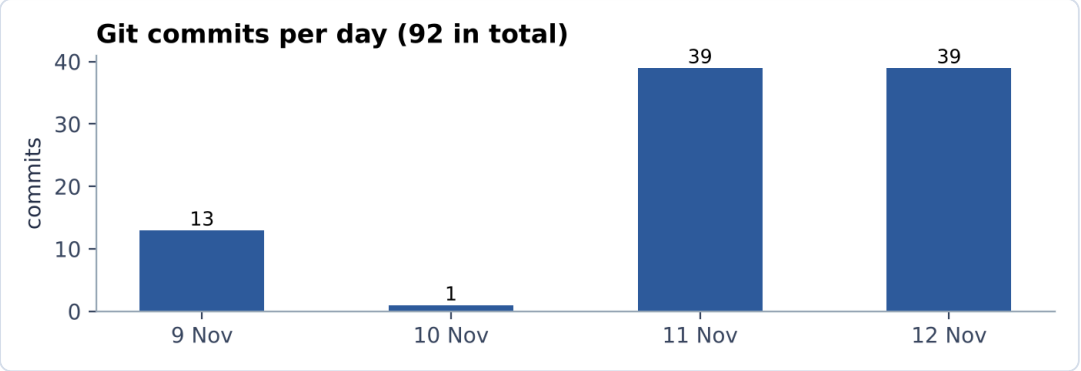


Figure 18. 92 commits on GitHub between 9 and 12 November 2026.

Phase	Main work
1 · Core (9 Nov)	PDF → passages, TF-IDF / BM25 search, NB / LR classifier, practice with hints, progress, evaluation script, logs.xlsx, dashboard, Arabic, dark mode, second course, doctor accounts
2 · Accounts and content (10–11 Nov)	Username / e-mail sign-in, registration, exams from question banks, translation, materials, exports, student home, course choice per student, calendar, notifications, profile with photo
3 · Studio and AI tools (11 Nov)	Study studio (videos, quiz, mind map), LSA / hybrid / neural search, automatic evaluation for any course, Colab notebooks, image recognition, OCR of picture slides, students at risk
4 · Staff tools and polish (12 Nov)	Chat requests and blocking, register steps with doctor approval, sub-admins with permissions, badges with creator, saved answers, study plan, announcements, weekly report, activity log, back arrow, responsive pass
5 · Model improvement (12 Nov)	Bigger test sets, query variants, more labels, leak-free threshold, direct Colab upload

15 Limitations and future work

15.1 Known limitations

- **Word matching:** TF-IDF and the Not-found gate work on words, so a paraphrase with no shared word can be missed, and an off-topic question that shares words can pass (3 of 50).
- **Picture-only slides** (worked examples drawn as images) have little text unless OCR or the Colab figure text is available.
- **Team-written questions:** the labelled and test questions were written by the team, not collected from real students; real numbers are usually lower.
- **Excel as a database** is simple and required by the sheet but not suited to many simultaneous users.

15.2 Real user test (to run before submission)

Five students each write 10 real questions and 2 off-topic questions and vote 👍 / 👎 on each answer; the lecturer reviews one weekly report. Results go into the sheet in `docs/USER_TEST.md`: right slide in the top three (of 50), off-topic refused (of 10), helpful votes and interface notes. Failed questions are added to `question_variants.csv` only *after* the test.

15.3 Future work

- Make the neural method the default on a host that allows the model download, if the Colab result confirms it.
- A small language model that rephrases the quick answer using only the retrieved slides and their sources.
- SQLite instead of Excel for the records (bonus in the sheet).
- Collect real student questions through the site and retrain monthly.

16 Submission package and how to run

16.1 Contents of the ZIP

```
StudyMate/
├── app.py                Flask application (python app.py)
├── config.py             paths and settings
├── requirements.txt      packages (+ requirements-optional.txt)
├── core/                 search, classifier, mastery, topics, logger (log_to_excel), …
├── scripts/              build_passages.py · train.py · evaluate.py
├── data/AI/              lectures/*.pdf + all CSV files + data_card.md
├── data/MAM336/          second course
├── models/               trained models (.joblib)
├── static/reports/       metrics.json, confusion matrices, threshold plot
├── templates/  static/   pages, styles, scripts
├── notebooks/            2 Colab notebooks + results of the stored run
├── tests/                92 automatic tests
├── docs/                 this report, presentation outline, user test, deploy guide
└── logs.xlsx             Questions · Answers · Visits · Feedback · Students · Exams
```

16.2 How to run

```
python -m venv venv
venv\Scripts\activate            # Windows   (source venv/bin/activate on Linux / macOS)
pip install -r requirements.txt
python app.py                    # open http://127.0.0.1:5000

# optional: rebuild everything from the PDFs
python scripts/build_passages.py
python scripts/train.py
python scripts/evaluate.py
python -m pytest -q              # 92 tests
```

The main admin signs in with the lecturer password set in `config.py` or the `LECTURER_PASSWORD` environment variable.

> **Summary.** Every required part of Project 1E is built and measured, the two bonus items deployment and Arabic support are done, and the numbers are reported on held-out data. The remaining step is the real user test with the lecturer and five students.